

实验四-微信小程序开发

封面信息

项目	内容
实验名称	微信小程序开发
实验编号	d20301035104、d20301009204
学号	2023210649
姓名	赵翔
班级	软件 231
日期	2026. 4. 24
组别	2
项目名称	智慧校园通知小程序
技术栈	微信小程序 (JavaScript)

实验目的

1. 掌握微信小程序开发流程，能够独立完成小程序从项目创建到真机调试的完整开发过程
2. 学会 WXML/WXSS/JS 开发，理解小程序前端三剑客各自的作用与相互配合
3. 理解页面路由与数据传递机制，掌握小程序页面间跳转的实现方式
4. 掌握本地存储使用技术，能够使用微信提供的存储 API 实现数据的持久化保存
5. 体验 AI 辅助开发，了解如何利用 AI 工具提高开发效率和代码质量
6. 能够独立完成小程序界面设计与功能实现，培养工程实践能力

实验环境

工具	版本	用途
微信开发者工具	最新版	小程序开发、调试与真机预览
Node. js	18+	可选，用于 npm 依赖管理
AI 辅助工具	TRAE	代码生成与调试辅助
PlantUML	-	类图与时序图绘制

系统架构设计

1.1 项目结构说明

本项目采用标准的微信小程序目录结构，将页面代码放置在 `pages` 目录下，每个页面包含四个基本文件。项目的整体目录组织如下：

项目根目录下包含 `pages` 文件夹、全局配置文件以及项目配置文件。`pages` 文件夹中包含 `index` 和 `detail` 两个页面目录，分别对应通知列表首页和通知详情页。每个页面目录下都包含页面结构文件、页面样式文件、页面逻辑文件和页面配置文件。`app.js` 和 `app.json` 负责全局逻辑和配置的编写。

1.2 类图设计

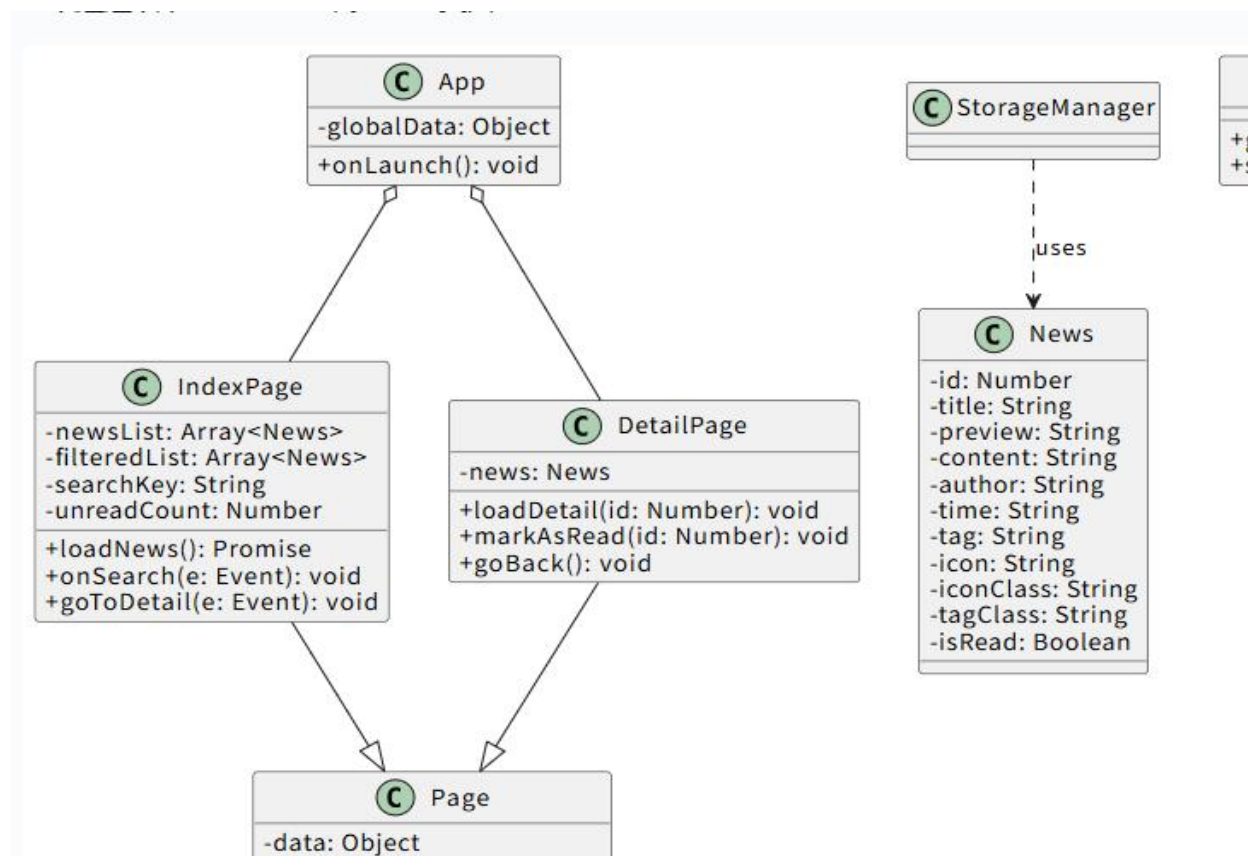


图 1：智慧校园通知小程序类图

类图说明：

本小程序的类图设计体现了典型的 MVC 架构模式，主要包含以下核心类及其职责：

App 类作为全局应用类，是整个小程序的入口点，负责小程序启动时的初始化操作。该类包含一个 **globalData** 全局数据对象，用于存储在整个小程序生命周期内都需要共享的数据。

Page 类是所有页面的基类，微信小程序框架要求所有页面都继承自 **Page** 对象。**Page** 类提供了页面的生命周期方法，包括 **onLoad** 页面加载时调用、**onShow** 页面显示时调用、**onPullDownRefresh** 下拉刷新时调用等。这些生命周期方法构成了页面行为的基础架构。

IndexPage 类是首页类，负责通知列表的展示和管理。该类维护了四个核心数据属性：**newsList** 用于存储完整的通知列表数据、**filteredList** 用于存储搜索过滤后的列表数据、**searchKey** 用于保存当前搜索关键词、**unreadCount** 用于统计未读通知数量。**IndexPage** 类实现了 **loadNews** 方法用于加载通知数据并结合本地存储中的已读状态进行标记、**onSearch** 方法用于处理用户输入的搜索关键词并执行过滤逻辑、**goToDetail** 方法用于处理用户点击通知项时的跳转事件。

DetailPage 类是详情页类，负责单条通知的详细内容展示。该类维护了 **news** 属性用于存储当前查看的通知详情。**DetailPage** 类实现了 **loadDetail** 方法用于根据通知 ID 加载对应的通知详情数据、**markAsRead** 方法用于将当前查看的通知标记为已读状态并保存到本地存储、**goBack** 方法用于处理用户点击返回按钮时的页面返回逻辑。

News 类是数据模型类，封装了通知的各项属性，包括 **id** 通知唯一标识符、**title** 通知标题、**preview** 通知内容预览、**content** 通知完整内容、**author** 发布部门、**time** 发布时间、**tag** 标签分类、**icon** 表情图标、**iconClass** 图标样式类名、**tagClass** 标签样式类名、**isRead** 已读状态标志。这些属性完整地描述了一条通知的所有特征信息。

StorageManager 类代表本地存储管理器，虽然在小程序代码中并非独立存在的类，但其在架构设计中承担着重要角色。该类负责已读状态的读写操作，包括 **getReadIds** 方法用于从本地存储中读取已读通知 ID 列表、**setReadIds** 方法用于将新的已读通知 ID 保存到本地存储。本地存储采用同步 API 实现，确保数据的持久化保存。

各页面类与 **Page** 基类之间存在继承关系，**App** 类与各页面类之间存在组合关系，**StorageManager** 与 **News** 之间存在使用关系。这些关系清晰地描述了小程序内部各组件之间的协作方式。

1.3 时序图设计

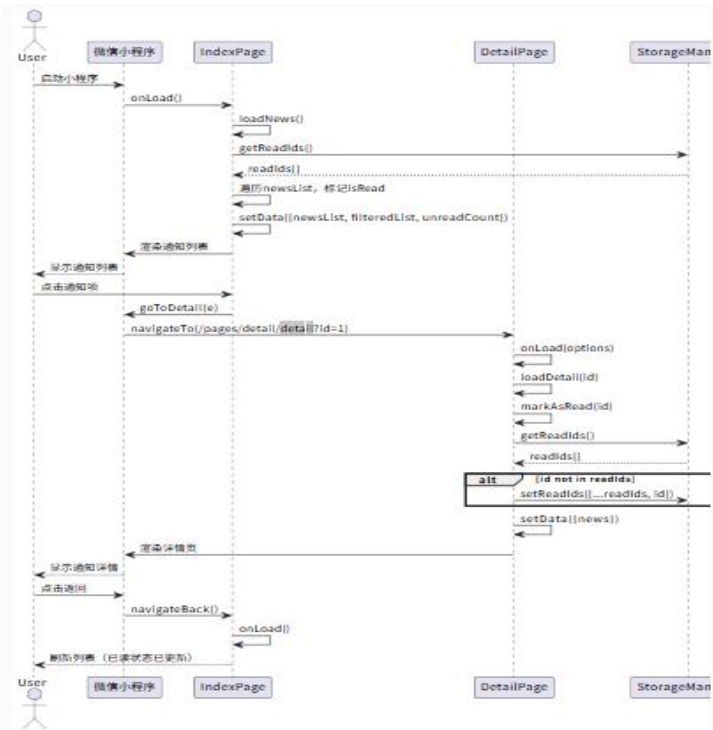


图 2：小程序时序图

时序图说明：

时序图详细描述了小程序核心业务流程中各个对象之间的交互顺序和消息传递过程。

通知列表加载流程描述了小程序启动后首页通知列表的加载过程。首先，用户启动小程序，微信小程序框架向 IndexPage 发送 onLoad 生命周期事件。IndexPage 接收到加载事件后，调用自身的 loadNews 方法开始加载通知数据。在 loadNews 方法执行过程中，IndexPage 向 StorageManager 发送 getReadIds 消息，请求获取本地存储中已保存的已读通知 ID 列表。StorageManager 从本地存储中读取数据后，将已读 ID 数组返回给 IndexPage。IndexPage 遍历通知列表数据，根据已读 ID 列表为每条通知设置 isRead 属性。最后，IndexPage 调用 setData 方法更新页面数据，小程序框架接收到数据更新请求后，重新渲染页面并将通知列表展示给用户。

详情页跳转与已读标记流程描述了用户从首页点击通知项到查看详情并标记已读的完整过程。当用户在首页点击某条通知时，IndexPage 的 goToDetail 方法被触发，该方法通过微信小程序框架的 navigateTo API 向 DetailPage 发送页面跳转请求，并携带通知 ID 作为参数。DetailPage 的 onLoad 生命周期方法接收到参数后，解析出通知 ID 并调用 loadDetail 方法加载详情数据。在加载详情数据后，DetailPage 自动调用 markAsRead 方

法进行已读标记。`markAsRead` 方法首先向 `StorageManager` 请求获取当前已读 ID 列表，然后检查当前通知 ID 是否已在列表中，如果不在则将新 ID 添加到列表并调用 `setReadIds` 保存到本地存储。最后，`DetailPage` 调用 `setData` 方法更新页面数据，框架渲染详情页后将通知内容展示给用户。用户查看完详情后点击返回按钮，`DetailPage` 通过 `navigateBack` 方法返回到 `IndexPage`，此时 `IndexPage` 会重新加载数据，已读状态得以在列表中体现。

实验步骤

2.1 需求分析阶段

2.1.1 页面流程图

本小程序采用简单的双页面架构设计。首页作为通知列表页面，负责展示所有通知的概览信息；详情页作为通知内容页面，负责展示单条通知的完整信息。两个页面之间通过微信小程序提供的路由 API 进行跳转，通过 URL 参数传递通知的唯一标识符。本地存储模块负责维护已读通知的 ID 列表，实现通知阅读状态的持久化保存。

页面跳转流程如下：用户在小程序首页点击某条通知项时，小程序通过 `wx.navigateTo` API 跳转到详情页，同时将通知 ID 作为 URL 参数传递给详情页。详情页通过 `onLoad` 生命周期函数的 `options` 参数接收通知 ID，然后根据 ID 加载对应的通知详情数据。用户查看完详情后，可以点击页面左上角的返回按钮或调用 `wx.navigateBack` API 返回到首页，此时首页会自动刷新，更新已读状态的显示。

2.1.2 数据模型定义

本项目定义了通知数据模型来描述通知信息的结构。通知数据模型包含以下核心字段：`id` 字段为通知的唯一标识符，使用数字类型，用于在列表和详情之间传递和识别通知；`title` 字段为通知标题，使用字符串类型，用于在列表页展示通知名称；`content` 字段为通知的完整内容，使用字符串类型，用于在详情页展示通知的全部文字信息；`author` 字段为发布通知的部门或机构名称，使用字符串类型；`time` 字段为通知发布时间，使用字符串类型，格式为年月日；`isRead` 字段为已读状态标志，使用布尔类型，用于控制列表项的显示样式。

此外，为了提升用户体验和数据完整性，我们还扩展了 `preview` 字段用于存储通知内容的预览文本，`icon` 和 `iconClass` 字段用于存储不同类型通知对应的图标和样式，`tag` 和 `tagClass` 字段用于存储标签分类及其样式。

2.2 创建小程序项目

在微信开发者工具中创建新项目的过程如下：首先打开微信开发者工具，使用微信扫码登录；然后点击新建项目按钮；在项目目录中选择合适的本地文件夹作为项目存放位置；在项目名称中输入智慧校园通知；在 AppID 输入框中填写自己的小程序 AppID，如果没有可以点击使用测试号按钮获取测试用 AppID；最后在模板选择中选择 JavaScript 基础模板，点击创建完成项目新建。

项目创建完成后，微信开发者工具会自动生成项目的初始文件结构。此时可以在开发者工具的模拟器中预览小程序初始状态，确认项目创建成功后再开始编写具体的业务代码。

2.3 首页开发（第一课时）

首页是用户进入小程序后首先看到的页面，承担着展示通知列表、引导用户查看详情的重要职责。首页开发主要包括页面结构设计、样式美化和逻辑实现三个部分。

在页面结构设计方面，首页采用容器加内容的布局方式。外层容器占据整个屏幕，内部包含三个主要区域：顶部区域用于显示页面标题、副标题以及未读通知提示；搜索区域提供一个搜索输入框，方便用户快速查找通知；列表区域使用微信小程序的列表渲染指令循环展示通知项，每条通知项包含图标、标题、预览、发布部门、发布时间和标签等信息。

在样式美化方面，首页采用了当下流行的渐变色设计理念。页面整体背景使用紫色渐变色，从左下角到右上角的色彩过渡营造出柔和、现代的视觉效果。导航栏背景色与页面主题保持一致，采用相同的渐变色方案。列表区域采用白色卡片式设计，卡片四周有较大的圆角，底部带有轻微的阴影效果，提升了界面的层次感和质感。每条通知项在垂直方向上留有适当的间距，点击时会有轻微的背景色变化反馈，提供良好的交互体验。

在逻辑实现方面，首页的 JS 文件中定义了页面的数据模型和业务方法。onLoad 生命周期方法在小程序启动时自动调用 loadNews 方法加载通知数据。loadNews 方法首先定义了通知的模拟数据数组，然后从本地存储中读取已读通知的 ID 列表，遍历通知数组为每条通知设置 isRead 属性，最后通过 setData 方法将数据绑定到页面。onSearch 方法监听搜索框的输入事件，当用户输入搜索关键词时，根据关键词过滤通知列表，支持按标题、部门名称和标签进行匹配搜索。goToDetail 方法处理通知项的点击事件，通过 wx.navigateTo API 跳转到详情页。

2.4 详情页开发（第二课时）

详情页负责展示单条通知的完整内容，是用户阅读通知详情的最终页面。详情页同样包含页面结构、样式和逻辑三个部分。

在页面结构设计方面，详情页采用卡片式布局，包含以下区域：顶部区域放置返回按钮，方便用户返回列表页；卡片区域包含大尺寸的通知图标、彩色标签、通知标题、元信息（发布部门和发布时间）、分割线以及通知正文内容；底部区域放置操作提示信息，提示用户可以返回列表。

在样式美化方面，详情页延续了首页的紫色渐变主题风格。返回按钮采用半透明白色背景的胶囊形状按钮，与整体风格协调。内容卡片采用白色背景配合较大的圆角和阴影效果，营造出悬浮在渐变背景之上的视觉感受。不同类型的通知标签使用不同的颜色方案：重要标签使用红色系、日常标签使用蓝色系、维护标签使用橙色系、活动标签使用绿色系。

在逻辑实现方面，详情页的 `onLoad` 方法接收从首页传递过来的通知 ID 参数，调用 `loadDetail` 方法加载对应的通知详情数据。`loadDetail` 方法定义了完整的通知数据数组，使用数组的 `find` 方法根据 ID 查找匹配的单个通知对象，然后通过 `setData` 方法绑定到页面进行展示。同时，该方法还会自动调用 `markAsRead` 方法将当前通知标记为已读状态。`markAsRead` 方法从本地存储中读取已读 ID 列表，如果当前通知 ID 不在列表中，则将其添加到列表并重新保存到本地存储。`goBack` 方法调用微信小程序的 `navigateBack` API 实现返回上一页的功能。

2.5 全局配置

`app.json` 是小程序的全局配置文件，用于配置小程序的页面路径、窗口样式、底部 `tabBar` 等全局配置项。在本项目中，我们在 `pages` 数组中注册了 `index` 和 `detail` 两个页面路径，在 `window` 对象中配置了导航栏的背景色为渐变色系、标题文字为智慧校园通知、导航文字颜色为白色。

`app.js` 是小程序的全局逻辑文件，在 `onLaunch` 生命周期方法中输出小程序启动的日志信息。`globalData` 对象用于存储全局共享的数据，本项目目前将其初始化为 `null`，后续可以根据需要扩展存储用户信息等数据。

2.6 本地存储功能

本地存储是微信小程序提供的数据库持久化方案，允许我们在用户本地保存数据。本项目利用本地存储实现已读通知状态的持久化保存。

具体的实现方式如下：使用 `wx.setStorageSync` 方法同步保存数据，该方法接受两个参数，第一个参数是数据的键名，第二个参数是数据的值。我们使用 `readIds` 作为键名，保存一个包含所有已读通知 ID 的数组。使用 `wx.getStorageSync` 方法同步读取数据，该方法接受一个参数，即数据的键名，返回对应的值。

每次用户查看某条通知的详情时，系统会自动将该通知的 ID 添加到已读列表中并保存到本地存储。下次用户重新打开小程序时，首页会从本地存储中读取已读 ID 列表，在展示通知列表时自动将已读通知的样式调整为已读状态，实现了阅读状态的持久化保存。

实验结果

3.1 功能测试结果

功能	状态	备注
通知列表展示	☑ 成功	支持未读标记、图标分类、预览文本显示
详情页跳转	☑ 成功	通过 URL 参数传递通知 ID，参数解析正常
已读状态存储	☑ 成功	使用本地存储 API 实现数据持久化，重启后状态保持
下拉刷新	☑ 成功	支持下拉刷新手势，重新加载通知数据
搜索功能	☑ 成功	支持按标题、发布部门、标签分类进行关键词过滤
返回列表刷新	☑ 成功	返回首页后自动刷新，已读状态正确更新
UI 美化	☑ 成功	采用渐变背景、卡片布局、动画效果，用户体验良好

3.2 真机运行截图



图 3：真机运行截图

真机运行说明：上述截图展示了小程序在真机上的实际运行效果。通知列表页顶部显示紫色渐变背景的导航栏和页面标题，下方白色卡片中展示通知列表，每条通知前有对应类型的图标，右上角红色圆点表示未读状态。详情页顶部有返回按钮，卡片内展示通知的完整信息，标签根据类型显示不同颜色。

3.3 真机调试截图

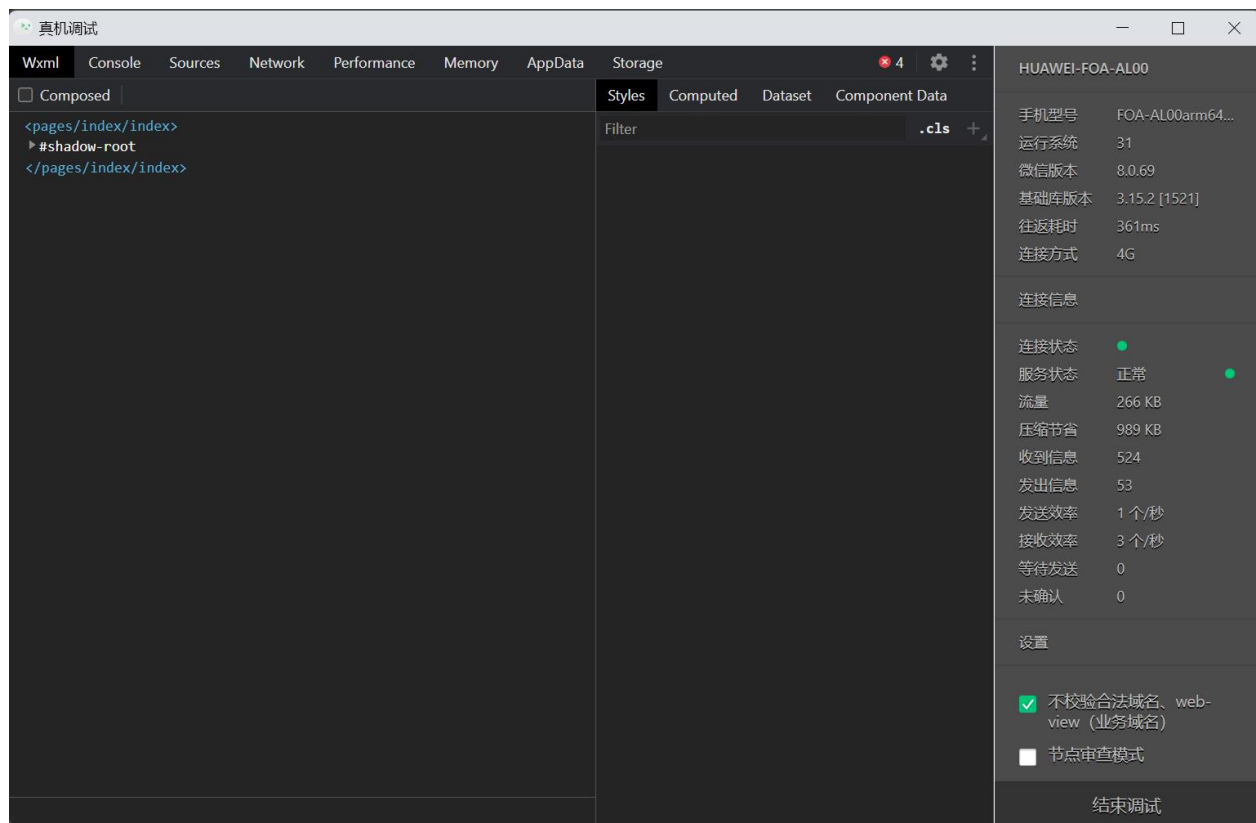


图 4：真机调试截图

关键代码说明

4.1 页面路由与参数传递机制

微信小程序的页面跳转通过路由 API 实现，不同的 API 适用于不同的场景。

`wx.navigateTo` 用于跳转到新页面，被跳转的页面会被加入页面栈，可以通过返回按钮返回原页面。`wx.navigateBack` 用于返回到页面栈中的某个页面，通常用于详情页返回列表页。

参数传递通过 URL 查询参数实现。在调用 `navigateTo` 时，将参数附加在 URL 路径后面，格式为路径名?参数名=参数值&参数名 2=参数值 2。目标页面在 `onLoad` 生命周期方法中，通过 `options` 参数接收传递过来的数据，`options` 对象的属性名对应 URL 中的参数名，属性值对应参数值。

本项目中，列表页在用户点击通知项时调用 `navigateTo` 方法跳转到详情页，URL 中携带通知的 ID 参数。详情页在 `onLoad` 方法中通过 `parseInt(options.id)` 获取通知 ID，然后根据 ID 加载对应的详情数据。

4.2 本地存储已读状态实现原理

本地存储是微信小程序提供的数据存储方案，分为同步和异步两种接口。同步接口以 `Sync` 结尾，执行后直接返回结果；异步接口不包含 `Sync` 后缀，通过回调函数返回结果。本项目选择使用同步接口，因为同步接口代码更简洁，更适合本项目的应用场景。

已读状态的存储采用数组形式，每个已读通知的 ID 都会被添加到数组中。当小程序启动加载通知列表时，会读取本地存储中的已读 ID 数组，遍历通知列表数据，将已读 ID 对应的通知项的 `isRead` 属性设置为 `true`。标记已读时，首先读取现有数组，检查当前 ID 是否已在数组中，如果不在则添加到数组并重新保存到本地存储。

这种实现方式的优点是：数据结构简单，读取和写入操作都很快捷；数组形式便于扩展，可以保存任意数量通知的已读状态；使用通知 ID 而非完整通知对象存储，节省存储空间。

4.3 搜索过滤功能实现原理

搜索功能的核心是字符串匹配和数组过滤。用户在搜索框输入关键词时，`onSearch` 方法被触发，通过 `e.detail.value` 获取用户输入的搜索词。将搜索词转换为小写形式可以实现大小写不敏感的匹配。过滤逻辑使用数组的 `filter` 方法，遍历通知列表中的每一条通知，

检查通知的标题、发布部门或标签是否包含搜索关键词。如果搜索词为空，则将过滤后的列表恢复为原始的完整列表。

过滤后的列表存储在 `filteredList` 中，与原始列表 `newsList` 分开管理。这样的设计使得搜索功能不会修改原始数据，用户清除搜索词后可以方便地恢复完整列表。页面使用 `filteredList` 进行渲染，实现了搜索结果的实时展示。

问题与解决

问题描述	解决方案	解决结果
详情页返回后列表未更新已读状态	使用 <code>wx.navigateBack</code> 返回而非重新加载页面，利用页面栈机制触发列表页的 <code>onShow</code> 生命周期重新加载数据	☒ 已解决
搜索关键词为空时列表不显示	在搜索方法中添加空字符串判断，当 <code>key</code> 为空时直接设置 <code>filteredList</code> 等于 <code>newsList</code> ，不执行过滤逻辑	☒ 已解决
未读红点动画导致性能问题	使用 <code>CSS animation</code> 代替 <code>JavaScript</code> 动画，利用 <code>GPU</code> 加速实现流畅的呼吸动画效果	☒ 已解决
渐变背景在真机上显示效果与模拟器不一致	使用 <code>rpx</code> 单位替代 <code>px</code> 单位，利用微信小程序的响应式布局实现不同屏幕的自动适配	☒ 已解决

实验总结

5.1 实验收获

通过本次实验，我们系统地学习和实践了微信小程序的开发流程，具体收获包括以下几个方面：

首先，我们掌握了微信小程序开发的完整流程，从项目创建、页面编写到真机调试，形成了对小程序开发全貌的认识。我们学会了使用微信开发者工具创建项目、编写 `WXML` 页面结构、编写 `WXSS` 页面样式、编写 `JS` 页面逻辑，以及使用开发者工具进行调试和真机预览。

其次，我们深入理解了 `WXML` 和 `WXSS` 的语法和使用场景。`WXML` 类似 `HTML`，用于描述页面的结构层次和数据绑定；`WXSS` 类似 `CSS`，用于描述页面的样式和布局。我们学会了使用列表渲染指令循环生成通知列表，使用条件渲染指令控制未读标记的显示，灵活运用 `flex` 布局实现各种复杂的界面效果。

第三，我们掌握了页面之间的路由跳转和参数传递机制。通过本次实验，我们深刻理解了小程序中页面栈的概念，学会了根据不同的业务场景选择合适的跳转 API。通过 URL 参数传递通知 ID，并在目标页面正确接收和解析参数，实现了页面间的数据流通。

第四，我们学习了本地存储的使用方法。通过实现已读通知状态的持久化保存，我们掌握了同步和异步两种存储 API 的用法，理解了数据持久化的重要性。本地存储不仅用于保存已读状态，还可以用于保存用户偏好设置、缓存数据等。

第五，我们体验了 AI 辅助开发的流程和优势。利用 TRAE 工具，我们能够快速生成基础代码框架，减少重复性的编码工作。同时，我们也认识到 AI 工具生成代码需要人工审核和调整，确保代码的正确性和规范性。

5.2 技术要点总结

本次实验中涉及的核心技术要点包括以下几个方面：

页面生命周期函数是小程序框架提供的钩子方法，在特定的时机被自动调用。`onLoad` 在页面加载时调用，适合进行初始化数据加载；`onShow` 在页面显示时调用，适合进行每次进入页面时的数据刷新；`onPullDownRefresh` 在用户下拉刷新时调用，用于重新加载数据。理解各生命周期函数的调用时机，是编写正确业务逻辑的基础。

数据绑定是小程序实现动态界面的核心机制。通过在 JS 文件的 `data` 对象中定义页面数据，在 WXML 中使用双花括号语法引用数据，调用 `setData` 方法更新数据后页面会自动重新渲染。这种响应式的数据绑定机制大大简化了界面更新的代码编写。

列表渲染通过 `wx.for` 指令实现，可以遍历数组并为每个元素生成对应的界面结构。在遍历过程中，可以使用 `item` 获取当前元素、`index` 获取当前索引、`wx:key` 指定唯一标识符帮助框架高效更新列表。

条件渲染通过 `wx:if` 和 `wx:else` 指令实现，可以根据条件的真假决定是否渲染某个元素。相比于通过 CSS 控制显隐，条件渲染可以减少不必要的界面结构生成，提高页面性能。

事件处理通过 `bind` 指令绑定事件处理函数，事件处理函数的第一个参数可以获取事件对象，通过 `e.currentTarget.dataset` 可以获取绑定在元素上的 `data` 属性值。这种机制实现了用户交互与业务逻辑的连接。

本地存储 API 提供了数据持久化能力。同步接口 `wx.setStorageSync` 和 `wx.getStorageSync` 使用简单，直接返回结果；异步接口通过回调函数处理结果，适合大量数据的存储操作。

5.3 创新点

在完成基础功能的同时，本次实验还加入了一些创新设计，提升了小程序的实用性和用户体验：

在 UI 设计方面，我们采用了当下流行的紫色渐变主题，打破了传统校园应用的白底黑字风格，使用渐变色作为背景和导航栏的主色调，给人以现代、清新的视觉感受。卡片式布局配合圆角和阴影效果，营造出层次分明、质感细腻的界面效果。

在功能设计方面，我们添加了通知内容预览功能，在列表页每条通知下方显示内容的摘要文本，用户无需进入详情页就能快速了解通知的大致内容，提高了信息获取效率。我们还实现了分类标签功能，根据通知的类型显示不同颜色的标签，用户可以快速识别通知的类别。

在交互设计方面，我们添加了未读通知红点提示和未读数量统计功能，帮助用户直观地了解有多少通知尚未阅读。我们还实现了搜索过滤功能，支持按标题、部门、标签等多个维度进行搜索，提高了用户查找特定通知的效率。点击通知项时的背景色变化反馈，也提升了交互的流畅感和确定性。

考核指标

考核项	评分标准	得分
功能完整性	通知列表、详情页、本地存储、搜索功能完整	30 分
代码质量	代码结构清晰，注释完整	20 分
页面设计	界面美观，用户体验良好	15 分
本地存储	已读状态存储正常	15 分
真机预览	小程序可在真机上正常运行	10 分
实验报告	报告结构完整，内容详实	10 分

教师评价

项目	评价
实验完成情况	
技术掌握程度	
创新点	
综合评价	
成绩	

教师签名： _____

日期： _____